SEASUG Paper 55-2026

Streamlining Course Evaluation Reporting for Faculty Review Using PROC REPORT, ODS, and File Manipulation

Karma Brainard, BS candidate, Abbas S. Tavakoli, DrPH, MPH, ME, University of South Carolina, College of Nursing

ABSTRACT

Course evaluations provide valuable feedback; however, generating clear and accurate reports for individual faculty members and coordinators is both time-consuming and error prone. Raw course evaluation data is provided in CSV files that contain unclear variable names and duplicate records. This paper presents a streamlined solution that transforms these raw exports into standardized course and teacher evaluation reports using R for data preparation, followed by SAS 9.4 PROC REPORT and ODS for formatted output, along with automated folder and file manipulation. The approach generates collective reports and individual reports for each faculty member, all with consistent styling. The result is a reproducible workflow that can be executed each semester, reducing processing time, minimizing errors, and enhancing report readability.

INTRODUCTION

Course evaluation data is exported each semester as a CSV file from the online survey platform. The file contains all necessary response data, but it requires substantial preparation before the results can be reported to faculty. Individual reports are created for course evaluations, course comments, teacher evaluations, and teacher comments. Additional reports show overall means across course levels for both teachers and courses, along with summaries by course and section and by course and teacher.

Producing these reports used to be almost entirely manual. Each semester requires cleaning the CSV file (removing unnecessary rows and columns, renaming variables, and creating columns) before the data could be read into SAS. Then in SAS, enrollment figures were hard-coded, courses were combined, and duplicate observations were removed. The reports themselves were written using FILE PRINT and PUT statements, so any small change required rewriting code to keep the output aligned. Then the PDFs were split and sorted into folders by hand. The process was time-consuming, hard to edit, error-prone, and the SAS monospaced font was difficult to read.

8	class_type	Char	7	$7.
9	SecondarySubjectID	Char	9	$9.
10	Given_name	Char	9	$9.
11	Family_Name	Char	9	$9.
12	EnrollmentType	Char	6	$6.
13	FilloutDate	Num	8	DATETIME16.
14	Q1_A_response_is_required_for_Co	Num	8	BEST.
15	Q1_A_response_is_required_for_C1	Num	8	BEST.
16	Q1_A_response_is_required_for_C2	Num	8	BEST.
17	Q1_A_response_is_required_for_C3	Num	8	BEST.
18	Q1_A_response_is_required_for_C4	Num	8	BEST.
19	Q2_What_aspects_of_this_course_i	Char	2287	$2287.
20	Q3_A_response_is_required_for_Co	Num	8	BEST.
21	Q3_A_response_is_required_for_C1	Num	8	BEST.
22	Q3_A_response_is_required_for_C2	Num	8	BEST.
23	Q4_What_about_this_instructor_im	Char	878	$878.
24	VAR24	Char	3	$3.

Display 1. Format of original CSV file

DATA PREPARATION IN R

Before reading the data into SAS, an R script is used to prepare the original CSV export. The script renames columns, separates the teacher and course evaluation data, and removes duplicate records from the course evaluation dataset. Data preparation uses the readr, stringr, dplyr, and openxlsx packages. Two parameterized variables are used to allow the base directory and semester to be changed easily between evaluation periods.

After the original data is read into R, the desired columns are selected. Because the order and names of the exported columns frequently change between semesters, these steps are more easily handled in R prior to importing the data into SAS. The script uses the stringr package to split the Subject ID into separate course and section variables, including their numeric and character components.

Separate datasets are created for the course evaluation (ceq) and teacher evaluation (teq). Duplicates are removed from the ceq data using grouped filtering based on course, enrollment, and responses. The script also converts the comments to ASCII to prevent character encoding issues when importing the data into SAS. Then the two datasets are exported to Excel:

```
eval$ncrs <- str_extract(eval[["id"]],"(?<=NURS)\\w+(?=-)")
eval$crs <- as.numeric(str_extract(eval$ncrs,"\\d+"))
eval$crsc <- str_extract(eval$ncrs,"[A-Za-z]+")
cdat <- cdat %>%
  group_by(across(all_of(c("ncrs","nsec","enrol","ceq1","ceq2","ceq3","ceq4",
  "ceq5","ceqcom")))) %>% filter(n_distinct(fname) == 1 | fname ==
  first(fname)) %>% ungroup()

teq$teqcom <- iconv(teq$teqcom, from = "UTF-8", to = "ASCII//TRANSLIT")
write.xlsx(teq, teqfile, rowNames=FALSE)
```

DIRECTORY CREATION

The driver program in SAS begins by defining global macro variables that control semester settings, enrollment, report styling, and output locations. Using the DLCREATEDIR option, the program automatically creates semester-specific directory structures for each report type. Then macro variables are created to reference file names. The Excel files created in R are imported into SAS using macro variables defined in the driver program. The imported data is then filtered to create the final ceq and teq datasets. Records with enrollment below the minimum threshold are removed, along with any records that need to be excluded from the analysis:

```
%let semyear=Spring 2026;
%let sem=SP26;
%let minenrol=3;
%let withsec=208 220;
%let font=Times New Roman;
%let fsize=9pt;
%let titlefontsize=14pt;
%let colorfmt=cxD9D9D9;

libname mdir 'C:\Evals\';
%let basedir=%sysfunc(pathname(mdir));
options dlcreatedir;
libname edat "&basedir.\Data\&sem";
libname esem "&basedir.\Output\&sem";
libname eceq "&basedir.\Output\&sem.\CEQ-EVAL-&sem";
options nodlcreatedir;
```

```
%let ceqfile=%sysfunc(pathname(edat))\Ceq&sem..xlsx;
%let ceqevalpath=%sysfunc(pathname(eceq))\CEQ-EVAL-&sem-;

proc import datafile="&ceqfile" out=edat.ceq&sem dbms=xlsx replace;
    getnames=yes;
run;

data ceqdat;
    set edat.ceq&sem;
    if enrol < &minenrol then delete;
    if (ncrs="") then delete;
run;
```

REUSED MACROS

Several macros are used across multiple evaluation types. These macros combine the sections of courses not listed in the &withsec macro variable, calculate enrollment for course and teacher reports, count response categories for evaluation tables, and generate titles. To simplify file creation, the SORT procedure with the NODUPKEY option also creates two datasets that keep one observation for each unique combination of the sort variables. Enrollment is calculated with the SQL procedure:

```
%MACRO ENROL;
    proc sort data=&type.eqdat(keep=&sortvar enrol) nodupkey out=enroldis;
        by &sortvar enrol;
    run;

    DATA enroldis;
        set enroldis;
        %RMSEC;
    run;

    proc sql;
        create table &type.enrol as select
            %sysfunc(tranwrd(&sortvar., %str( ),%str(, ))),
            sum(enrol) as enrolf from enroldis group by
            %sysfunc(tranwrd(&sortvar., %str( ), %str(, )));
    quit;

    data &type.enrol;
        set &type.enrol;
        enrol=input(enrolf,best12.);
    run;
%MEND ENROL;
```

DATA MACROS

Eight report types are needed: means, summary, comment, and evaluation reports, each for teachers and for courses. As a result, the program has four macros to analyze the data and four macros to create the reports. These macros read in type as teacher or course. This allows for consistent reports between the two categories.

The macros that format the data for the reports use standard methods; therefore, the code has been omitted. The comment macros remove comments from a list of N/A answers and comments below a certain length. The evaluation, summary, and means macros use multiple MEANS procedures, merges, and an array DATA step to get the desired format for the REPORT procedures. The means macros also use multiple TRANSPOSE procedures. All data macros end in a SORT procedure by sortvar, the final dataset is then output to be fed into the report macro.

REPORT MACROS

Each report macro consists of a single REPORT procedure that takes its styling from global macro variables and its titles from a shared title macro. The comment and evaluation reports are split by the sort variables, and #BYVAL references in the titles display the teacher, section, course, and enrollment.

The evaluation report consists of one row per question with columns: question, five Likert response counts, total responses, class mean, class standard deviation, and college mean. The questionteq. format displays the question text in place of the question number, so questions can be changed easily by editing a single FORMAT procedure:

```
%MACRO EVALREPORT;
      proc report data=&type.eqe nowd split="|"
            STYLE(report)=[rules=none frame=hsides]
            STYLE(header)=[font_size=&fsize font_face="&font"
                        cellheight=.4in vjust=middle background=&colorfmt]
            STYLE(column)=[font_size=&fsize font_face="&font"
                        cellheight=.4in vjust=middle cellwidth=7%];

            by &sortvar enrol;
            column num a1 a2 a3 a4 a5 n clsmean clssd clgmean &sortvar enrol;

            define num / display format=question&type.eq. "Question" left
                  style(column)=[cellwidth=36%];
            define n / display "Total|Resp" format=8. center;
            define clgmean / display "College|Mean" format=8.2 center;
            define clssd / display "Class|Std Dev" format=8.2 center;
            define clsmean / display "Class|Mean" format=8.2 center;
            define a1 / display "Strongly Disagree" center;
            define a2 / display "Disagree" center;
            define a3 / display "Neutral" center;
            define a4 / display "Agree" center;
            define a5 / display "Strongly Agree" center;
            define ncrs / noprint;
            define nsec / noprint;
            define crs / noprint;
            define sec / noprint;
            define crsc / noprint;
            define secc / noprint;
            define enrol / noprint;
            %if &type.=T %then %do;
                  define lname / noprint;
                  define fname / noprint;
            %end;
            %reporttitles(evaltitle=&evaltitle);
            title4 " ";
            title5 font="&font" height=&headerfontsize j=c
                  "Course: #byval(ncrs)-#byval(nsec)";
            title6 font="&font" height=&headerfontsize j=c
                  "Enrollment: #byval(enrol)";
            %if &type.=T %then %do;
                  title7 font="&font" height=&headerfontsize j=c
                        "Teacher: #byval(fname) #byval(lname)";
            %end;
      run;
%MEND EVALREPORT;
```

CREATING PDFS

The reportpdf macro generates the PDF output. It first validates type, writing an error to the log if it has a value other than c or t. Next, it validates the method parameter, which must be evaluation, comments, summary, or means. The macro suppresses output to the log and stops PDFs from automatically opening. For clean, uniform output, it sets missing values to display as blank spaces and applies the same margins to every PDF. After the setup macro builds the dataset, reportpdf writes the output with multiple ODS PDF statements. For the evaluation and comments methods, PDFs are created for each level using NEWFILE=BYGROUP. Then another ODS PDF is run to create a master document of all levels. The summary and means methods skip BY processing and each create a single PDF.

```
%macro reportpdf(method=, type=,evaltitle=);
      %let method=%upcase(&method);
      %let type=%upcase(&type);
      %if (&type.=C or &type.=T) %then %do;
            %if (&method.=MEAN or &method.=SUM or &method.=EVAL or
            &method.=COM) %then %do;
                  %setvar;
                  %enrol;
                  %&method.set;

                  %local path;
                  %let path=&&&type.eq&method.path;
                  %hidelog;
                  %if (&method.=EVAL or &method.=COM) %then %do;
                        ods pdf file="&path.1.pdf" newfile=bygroup notoc;
                        %&method.report;
                        ods pdf close;

                        data _null_;
                              set &type.sort;
                              r=rename(cats("&path.",_n_,".pdf"),
                              cats("&path.",&type.name,".pdf"),"file");
                        run;
                  %end;

                  ods pdf file="&path..pdf" notoc;
                  %&method.report;
                  ods pdf close;


                  %showlog;
            %end;
            %else %do;
            %put Method is not valid. Use sum, mean, eval, or com;
            %return;
            %end;
    %end;
    %else %do;
        %put Type is not valid. Use c for ceq or t for teq;
        %return;
    %end;

%mend reportpdf;
```

In the main program, the reportpdf macro is called for each of the eight report types, with parameters for the type, the method and the evaluation title. Declaring them this way allows the user to see what is sent

to each macro from the main program. Similarly, evaltitle sends the evaluation name to the reporttitles macro, allowing for easy changes from the main document. This also labels what each macro does without comments:

```
%reportpdf(method=sum, type=c, evaltitle= Course Summary Evaluation);
%reportpdf(method=eval, type=t, evaltitle= Teacher Evaluation);
```

TEACHER FOLDERS

Folders are created for each teacher in the dataset. A DATA _NULL_ step is used to create folders for each teacher inside the faculty folder. To create folders for the coordinators, the dataset is first converted to long format, and then the SQL procedure creates a dataset that matches faculty coordinators to courses. Finally, two DATA _NULL_ steps read the teacher and faculty datasets and copy each matching file into its folder:

```
%macro distributeevals;
    data _null_;
        set tsort;
        r=dcreate(tchr, "&facdir");
    run;

    data _null_;
        set fc;
        r=dcreate(cordname, "&cordir");
        ndir=cats("&cordir",cordname);
        r=dcreate('CEQ-COM', ndir);
        r=dcreate('CEQ-EVAL', ndir);
    run;

    data fclong;
        set fc;
        do i=1 to countw(c1, ' ');
            crs=input(scan(c1, i, ' '), 3.);
            output;
        end;
        keep cordname crs;
    run;

    proc sql;
        create table fsort as select * from fclong, csort where
            fsort.crs=csort.crs;
    quit;

    data _null_;
        set tsort;
        length src dst $400;

        src=cats("&teqcompath", tname,'.pdf');
        dst=cats("&facdir.", tchr,"\TEQ-COM-",tname,'.pdf');
        %evalcopy();

        src=cats("&ceqcompath", cname,'.pdf');
        dst=cats("&facdir.", tchr,"\CEQ-COM-",cname,'.pdf');
```

```
        %evalcopy();

        src=cats("&teqevalpath", tname,'.pdf');
        dst=cats("&facdir.", tchr,"\TEQ-EVAL-",tname,'.pdf');
        %evalcopy();

        src=cats("&ceqevalpath", cname,'.pdf');
        dst=cats("&facdir.", tchr,"\CEQ-EVAL-",cname,'.pdf');
        %evalcopy();
    run;

%mend distributeevals;
```

SAMPLE REPORT

University of South Carolina
College of Nursing
Teacher Evaluation Spring 2026

Course: 717-000
Teacher:
Enrolment: 28

Question	Strongly Disagree	Disagree	Neutral	Agree	Strongly Agree	Total Resp	Class Mean	Class Std Dev	College Mean
This instructor treated students with respect and created an environment in which students felt encouraged to participate.	0	0	0	6	16	22	4.73	0.46	4.48
I was able to reach my instructor when needed, and they were responsive to my questions and concerns.	0	0	1	7	14	22	4.59	0.59	4.43
The instructor carefully explained difficult concepts, methods, and subject matter.	0	3	1	5	13	22	4.27	1.08	4.36
The instructor was available to students either electronically or in person.	0	0	0	7	15	22	4.68	0.48	4.44
The instructor showed a thorough knowledge of the course material.	0	0	1	4	17	22	4.73	0.55	4.44
Overall, the instructor was effective in teaching this course.	0	2	1	7	12	22	4.32	0.95	4.39
TOTAL: Based on questions 1-5	0	3	3	29	75	22	4.60	0.51	4.43

Display 2. Teacher Evaluation Report

The new process was used at the University of South Carolina College of Nursing for Spring 2026. The previous method required substantial time each semester for Excel formatting, code adjustment, and file sorting; the new method requires only updating a few variables and running one R script and one SAS program, which takes minutes rather than days. This is especially useful when a mistake or a late update requires the process to be rerun.

The new method not only saves time but also increases accuracy, improves appearance, and allows for easy edits. Enrollment was previously hard-coded, creating potential for error; the new code instead reads enrollment from the original file. The monospaced font in the old output was difficult to read, whereas the new program allows global edits to the font and size. It also makes simple edits much easier:

changing a question requires only an edit to a PROC FORMAT, and changing which courses have combined sections requires only a change to a global variable. Even larger changes are easier to implement because of the code's structure. For example, this program served as the base for the undergraduate and graduate faculty reviews, and recoding it for a different CSV format, set of questions, and folder structure took less than 30 minutes.

University of South Carolina
College of Nursing
Course Evaluation Summary Spring 2026

Course	Section	Enrollment	Total Resp	Class Mean	Class Std Dev
208	N10	139	79	3.72	0.89
208	N11	139	70	3.71	1.02
208	N12	140	82	4.08	0.89
208	N13	135	71	3.89	0.94
220	001	142	70	4.85	0.34
220	H01	28	13	4.23	0.86
220	N10	179	50	4.47	0.59
318	000	5	4	4.75	0.50
400	000	164	56	4.13	0.99
411	000	155	51	3.73	1.06
412	000	162	57	3.64	1.11
422	000	155	77	4.85	0.33
424	000	158	64	4.27	1.03
425	000	162	101	4.45	0.87

Display 3. Teacher Summary Report

CONCLUSION

This approach leverages the strengths of both R and SAS by using R for data preparation and SAS macros and procedures for analysis and report generation. R's efficient tools for renaming and creating variables, as well as filtering and grouping data, simplify the cleaning of raw CSV files. SAS, through PROC REPORT and the Output Delivery System (ODS), produces professional, well-formatted reports. The use of parameterized macros and global macro variables creates consistent output that is simple to change. Together, these tools create an efficient and reproducible workflow for processing and reporting course evaluation data.

REFERENCES

SAS Institute Incorporated. (2013). SAS for Windows 9.4. Cary, NC: SAS Institute Inc.

SAS Support Communities. 2023. "Copy SAS Dataset Files from One Directory to Another." Accessed October 8, 2026. https://communities.sas.com/t5/SAS-Programming/Copy-sas-dataset-files-from-one-directory-to-another/td-p/876177.

CONTACT INFORMATION

Your comments and questions are valued and encouraged. Contact the author at:

Abbas S. Tavakoli, DrPH, MPH, ME
Clinical Professor
College of Nursing
University of South Carolina
1601 Greene Street

Columbia, SC 29208-4001
Fax: (803) 777-5561
E-mail: abbas.tavakoli@sc.edu

Karma Brainard
E-mail: karmab@email.sc.edu

SAS and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration.
Other brands and product names are trademarks of their respective companies.